Plano de Gerenciamento da Qualidade

Projeto Brigadeiro · Documento de planejamento acadêmico · Versão 1.0 · 10 de outubro de 2026

Equipe: Alice, Carolina, Luísa e Maria Luiza

Referencial: boas práticas de gerenciamento de projetos e qualidade, orientadas pelo PMBOK. Referências complementares de qualidade e testes de software são indicadas ao final.

Nota de aplicação
Este é um plano proposto para um MVP em desenvolvimento. As metas quantitativas são hipóteses iniciais a validar com a equipe e com usuários; não representam resultados medidos. Como ainda não há testes implementados, as atividades descritas são planejadas, não executadas.

1. Objetivo

Estabelecer como a equipe planejará, assegurará, controlará e melhorará a qualidade do Brigadeiro durante o desenvolvimento do MVP e de suas evoluções. O plano define critérios de aceitação, métricas, práticas de verificação, tratamento de defeitos, evidências e condições mínimas para considerar uma entrega pronta.

2. Contexto e princípios de qualidade

O Brigadeiro é um assistente de planejamento de refeições e gestão de ingredientes, pensado para reduzir o esforço de decidir o que preparar e facilitar o uso dos alimentos disponíveis. A experiência deve ser prática, fácil de entender e adequada ao celular. A qualidade será uma responsabilidade compartilhada, integrada ao desenvolvimento.

- Foco no usuário: validar se as funcionalidades resolvem necessidades reais e se os fluxos são compreensíveis.
- Prevenção: revisar requisitos, alterações e código para reduzir defeitos antes da entrega.
- Verificação baseada em risco: priorizar fluxos críticos, dados do usuário e integrações.
- Rastreabilidade: relacionar requisito, critério de aceitação, teste, defeito e versão no GitHub.
- Melhoria contínua: usar evidências de testes e feedback para rever prioridades e metas.

3. Escopo do produto e classificação dos requisitos

As funcionalidades discutidas são registradas abaixo, sem presumir que já estejam implementadas. A classificação inicial deve ser confirmada pela equipe no backlog do GitHub e revista quando o escopo mudar.

Funcionalidade / requisito	Classificação inicial	Foco de qualidade e critério proposto
Registro de ingredientes por texto	MVP	Usuário insere, revisa e salva ingrediente; campos obrigatórios são validados e erros orientam a correção.
Registro por áudio	MVP / implementação a confirmar	Quando disponível, a fala vira texto revisável antes da confirmação; falhas de captura não podem salvar dados incorretos silenciosamente.
Sugestões de refeições com base nos ingredientes disponíveis	MVP	Recomendação considera ingredientes informados e explica quais itens utiliza; ausência de resultados tem tratamento claro.
Alternativas de ingredientes (ex.: pão comum ou integral)	Requisito a validar	Alternativas são apresentadas sem alegar equivalência nutricional não verificada; usuário pode comparar e escolher.
Benefícios nutricionais das sugestões	Requisito a validar	Informações consistentes e sustentadas por fontes confiáveis; evitar alegações absolutas de saúde.
Filtro por tempo de preparo	MVP / prioridade a confirmar	Resultados respeitam o limite selecionado e exibem tempo estimado; regra de contagem deve ser documentada.
Foto de nota de mercado / OCR	Evolução futura	Itens extraídos são apresentados para revisão antes de salvar; valores incertos não são tratados como confirmados.
Experiência prática inspirada em apps de delivery (fotos, clareza, baixo atrito)	Requisito transversal	Ação principal é fácil de localizar; imagens correspondem à refeição e não prejudicam carregamento ou legibilidade.

Funcionalidade / requisito	Classificação inicial	Foco de qualidade e critério proposto
Integração com aplicativos de delivery	Evolução futura	Se aprovada, define-se escopo, tratamento de indisponibilidade e transparência sobre redirecionamento, preço e disponibilidade.
Uso móvel com entrada por texto, áudio ou imagem	Transversal / por fase	Interface funciona em telas móveis comuns, apresenta estados de carregamento e erro e evita perda de dados confirmados.

Regra de escopo: qualquer funcionalidade sem decisão formal permanece como “a validar”; não será obrigatória para aceitação do MVP até ser priorizada. Mudanças devem ser registradas no GitHub, incluindo impacto em qualidade e testes.

4. Objetivos e política da qualidade

A equipe buscará entregar incrementos úteis, seguros, compreensíveis e verificáveis. Uma funcionalidade só será considerada concluída quando atender aos critérios acordados, tiver sido revisada e testada de acordo com seu risco, não apresentar defeitos críticos conhecidos e possuir evidências mínimas registradas.

- Não marcar uma tarefa como concluída apenas porque o código foi escrito.
- Não aceitar dados de ingredientes ou resultados de OCR sem revisão quando houver incerteza relevante.
- Não publicar afirmações nutricionais sem base verificável.
- Tratar falhas que bloqueiam o fluxo principal ou causam perda/corrupção de dados como bloqueadoras.

5. Papéis e responsabilidades

A responsabilidade pela qualidade é coletiva e compartilhada entre Alice, Carolina, Luísa e Maria Luiza. A divisão operacional pode variar por tarefa; todas devem participar, conforme disponibilidade e contexto, da definição de critérios, revisão, testes e melhoria. O GitHub será o registro central do trabalho.

Atividade	Responsabilidade compartilhada / evidência mínima
Definir requisitos e critérios	Registrar descrição, prioridade, critérios de aceitação e dúvidas em issue ou artefato correspondente.
Implementar e revisar	Quem implementa solicita revisão de outra integrante sempre que viável e registra decisões relevantes.
Testar e registrar defeitos	Executar testes adequados ao risco; abrir issue com passos de reprodução e resultados esperado e obtido.
Aceitar entrega	Confirmar critérios, testes, defeitos remanescentes e limitações conhecidas antes de encerrar a tarefa.
Revisar o PGQ	Reavaliar metas, escopo, critérios e processo quando surgirem novas funcionalidades, falhas recorrentes ou feedback relevante.

6. Planejamento da qualidade e critérios de entrada/saída

6.1 Antes de iniciar uma funcionalidade

- Requisito descrito no GitHub e classificado por prioridade/fase.
- Critérios de aceitação observáveis e exemplos de uso, incluindo estados de erro quando aplicável.
- Riscos identificados: interpretação de áudio, qualidade de dados, privacidade, acessibilidade, desempenho e dependências externas.
- Evidências necessárias para demonstrar a conclusão definidas.

6.2 Para considerar uma funcionalidade concluída

- Todos os critérios de aceitação foram verificados e o resultado está registrado.
- Testes relevantes executados; falhas corrigidas ou registradas com decisão explícita.
- Código revisado por outra integrante sempre que viável e alteração versionada no GitHub.
- Nenhum defeito crítico conhecido sem mitigação ou decisão documentada.
- Documentação, instruções de uso e limitações atualizadas quando necessário.

7. Garantia da qualidade: prevenção

A garantia da qualidade verifica se o processo é adequado e seguido. A equipe deverá:

- Manter issues com escopo, prioridade, critérios de aceitação e vínculo com pull requests ou commits.
- Usar branches e pull requests para mudanças relevantes; evitar alterações diretas na branch principal quando possível.
- Revisar código com atenção à legibilidade, tratamento de erros, validação de entradas, segurança e efeitos colaterais.

- Manter instruções reproduzíveis para instalar, executar e testar o projeto.
- Revisar requisitos e protótipos antes de implementar fluxos complexos, especialmente áudio, OCR e recomendações.
- Revisar o processo quando um defeito escapar, ocorrer regressão ou houver retrabalho significativo.

8. Controle da qualidade: verificação do produto

Como ainda não existem testes implementados, os controles devem ser introduzidos incrementalmente. A seleção abaixo é uma proposta a adaptar à arquitetura e às tecnologias efetivamente utilizadas.

Tipo de teste	Aplicação proposta	Evidência
Testes manuais funcionais	Verificar entrada de ingredientes, consulta e recomendações; incluir campos vazios, entradas inválidas e ausência de resultados.	Checklist por requisito, resultado e versão testada.
Testes unitários	Testar validação, transformação de dados, filtros e regras de recomendação isoláveis.	Testes automatizados e resultado de execução.
Testes de integração	Verificar comunicação entre interface, backend, armazenamento e serviços de IA/áudio, conforme arquitetura real.	Casos de integração e registro de falhas.
Regressão	Reexecutar testes dos fluxos afetados após correções e mudanças relevantes.	Lista de testes reexecutados vinculada à alteração.
Usabilidade	Observar usuários representativos executando tarefas sem instrução excessiva; registrar dúvidas e erros.	Roteiro, observações e síntese sem dados pessoais desnecessários.
Acessibilidade e responsividade	Verificar contraste, tamanho de texto, rótulos, foco/teclado quando aplicável, mensagens de erro e telas móveis.	Checklist manual e issues.
Desempenho	Medir tempos dos fluxos principais em ambiente de teste definido, separando processamento local e serviços externos.	Medições com ambiente, amostra e versão.
Privacidade e segurança	Verificar segredos no repositório, validação de entradas e minimização/proteção de dados.	Checklist de revisão e correções registradas.

8.1 Ferramentas sugeridas

- GitHub Issues e Pull Requests: requisitos, tarefas, revisão, defeitos e rastreabilidade.
- pytest (se o backend usar Python): testes unitários e de integração.
- Playwright (se houver interface web compatível): automação de fluxos no navegador.
- Coverage.py (Python) ou ferramenta equivalente: observar quais partes do código são exercitadas, sem tratar cobertura alta como prova de qualidade.
- GitHub Actions: executar lint e testes em pull requests, quando o pipeline puder ser configurado.

As ferramentas são recomendações, não tecnologias confirmadas. A equipe deve escolher as que se encaixem na stack, no tempo disponível e no ambiente acadêmico.

9. Métricas e metas iniciais

As metas abaixo são propostas para iniciar a gestão e devem ser calibradas após obter uma linha de base com os primeiros testes e feedback. Cada medição deve registrar versão, ambiente, tamanho da amostra e método.

Indicador	Meta inicial proposta	Como medir / gatilho
Critérios de aceitação	100% dos critérios obrigatórios de uma tarefa verificados antes de aceitá-la.	Checklist vinculado à issue; cada entrega.
Testes automatizados	100% dos testes obrigatórios passam antes de integrar; falhas bloqueantes impedem integração até resolução.	Execução local/CI em cada pull request.
Defeitos críticos abertos	Zero defeitos críticos conhecidos no fluxo principal no momento da entrega.	Revisar issues antes da entrega e após regressões.
Revisão de código	Toda mudança relevante revisada por outra integrante, quando viável.	Pull requests; registrar exceções justificadas.
Sucesso em usabilidade	Pelo menos 80% dos participantes de rodada inicial concluem tarefas principais sem ajuda; validar amostra.	Teste por rodada; não generalizar amostras pequenas.
Tempo de resposta	Meta inicial: p95 abaixo de 3 s para operações locais/simples, excluindo serviços externos; revisar após linha de base.	Ambiente e rede documentados, com amostra suficiente.
Integridade do registro	100% dos casos de teste previstos preservam dados confirmados e informam erro para entradas inválidas.	Testes funcionais e de integração após mudanças de persistência.

Indicador	Meta inicial proposta	Como medir / gatilho
Rastreabilidade de defeitos	100% dos defeitos identificados têm issue ou registro equivalente com reprodução e severidade.	Revisar após cada rodada de teste.

A meta de tempo de resposta é um ponto de partida, não um compromisso validado. Para áudio, OCR e serviços externos, medir separadamente o tempo de processamento e a experiência percebida, pois a latência pode depender do fornecedor e da rede.

10. Severidade e tratamento de defeitos

Severidade	Definição	Tratamento
Crítica	Bloqueia fluxo principal, perde/corrompe dados, expõe informação sensível ou gera recomendação potencialmente prejudicial.	Interromper aceitação; priorizar correção/mitigação e repetir testes afetados.
Alta	Funcionalidade principal falha sem alternativa razoável ou erro recorrente afeta muitos usos.	Corrigir antes da entrega sempre que possível; exceção exige decisão e limitação documentadas.
Média	Problema funcional com alternativa disponível ou impacto limitado.	Registrar, priorizar e corrigir conforme risco; testar após correção.
Baixa	Problema visual ou de conveniência sem impacto relevante no fluxo principal.	Registrar no backlog e priorizar com outras melhorias.

Todo defeito deve registrar título claro, versão/ambiente, passos para reproduzir, resultado esperado e observado, severidade, evidência segura, próxima ação e estado. Após a correção, reexecutar o caso que falhou e testes de regressão relacionados.

11. Frequência e gatilhos dos controles

Quando / gatilho	Controle esperado
Ao criar ou alterar requisito	Revisar prioridade, critérios de aceitação, riscos, fase do escopo e casos de teste.
A cada mudança relevante de código	Executar verificações pertinentes e solicitar revisão por pull request quando viável.
Antes de integrar alteração	Executar testes disponíveis e revisar falhas; não integrar com falha crítica não resolvida sem decisão documentada.
Ao concluir funcionalidade	Executar checklist de aceitação, testes funcionais e revisão das evidências.
Após corrigir defeito	Reexecutar o teste que falhou e testes de regressão relacionados.
Antes de demonstração/entrega acadêmica	Executar fluxos principais de ponta a ponta e revisar defeitos críticos, limitações e instruções de execução.
Após feedback ou falha recorrente	Registrar achado, avaliar impacto, atualizar backlog e decidir se critérios, metas ou processo mudam.
Ao mudar escopo ou tecnologia	Reavaliar riscos, métricas, testes necessários e esforço de qualidade.

12. Gestão de mudanças e melhoria contínua

Mudanças no escopo, critérios de aceitação ou metas devem ser registradas no GitHub com justificativa e impacto esperado. A equipe revisará os resultados disponíveis, identificará defeitos recorrentes e ajustará o plano. Uma meta só deve mudar com justificativa e histórico preservado.

Quando um defeito importante escapar para demonstração ou entrega, a equipe deverá avaliar a causa provável — requisito ambíguo, teste ausente, revisão insuficiente, integração ou ambiente — e definir ação preventiva proporcional, não apenas corrigir o sintoma.

13. Riscos de qualidade e respostas

Risco	Resposta planejada
Requisitos mudam durante o MVP	Classificar fase e prioridade; atualizar critérios e testes vinculados à issue.
Recomendação inadequada ou pouco explicável	Definir exemplos esperados, testar casos comuns e limites, mostrar ingredientes utilizados e permitir corrigir dados.
Erros de áudio ou OCR	Apresentar resultado para revisão antes de confirmar; testar ruído, abreviações, baixa qualidade e campos ausentes.
Informação nutricional incorreta/exagerada	Usar fontes confiáveis, registrar origem e evitar alegações absolutas; manter como requisito a validar até definir verificação.
Dependência de serviços externos	Tratar timeout, indisponibilidade e limites de uso; comunicar falhas sem perder entradas confirmadas.
Pouco tempo para testes	Priorizar fluxos críticos e regressões; documentar limitações e não declarar cobertura inexistente.
Dados pessoais ou segredos expostos	Minimizar coleta, não versionar chaves, restringir acesso e remover dados sensíveis de evidências.

14. Registros e evidências de qualidade

Os registros devem permanecer acessíveis à equipe e vinculados ao repositório sempre que possível. Evidências mínimas:

- Issues para requisitos, riscos, defeitos e decisões de escopo.
- Pull requests, revisões e commits relacionados.
- Resultados de testes manuais e automatizados, incluindo versão e ambiente.
- Checklist de aceitação por funcionalidade ou entrega.
- Resultados de usabilidade e medições de desempenho, com método e limitações.
- Registro de exceções, defeitos aceitos temporariamente e justificativas.
- Histórico de mudanças do PGQ e das metas.

Não incluir senhas, tokens, dados pessoais desnecessários ou informações sensíveis em issues, logs, capturas de tela ou documentação pública.

15. Aprovação e revisão do plano

Este documento é uma proposta inicial. A equipe deverá confirmar a classificação do escopo, os critérios de aceitação, as metas quantitativas, a stack de testes e o fluxo de revisão antes de tratá-los como acordados. O PGQ deverá ser atualizado quando a arquitetura, as prioridades ou as evidências obtidas indicarem necessidade.

16. Referências

Project Management Institute (PMI). A Guide to the Project Management Body of Knowledge (PMBOK® Guide). Referência de princípios e práticas de gerenciamento de projetos; a edição específica deve seguir a orientação da disciplina.

Project Management Institute (PMI). Process Groups: A Practice Guide. Referência complementar para práticas de iniciação, planejamento, execução, monitoramento e encerramento, conforme abordagem adotada.

International Organization for Standardization (ISO). ISO/IEC 25010 — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model. Referência complementar para características de qualidade de produto de software; consultar a edição vigente aplicável.

International Organization for Standardization / International Electrotechnical Commission (ISO/IEC). ISO/IEC/IEEE 29119 — Software testing. Referência complementar para vocabulário e práticas de teste de software; consultar partes e edições vigentes pertinentes.

Nota bibliográfica: as referências normativas complementares são apoio técnico, não exigência adicional confirmada da disciplina. Antes da entrega acadêmica, conferir a edição efetivamente consultada e completar os dados bibliográficos exigidos pela instituição.